

Fixed Window Counter Algorithm (Rate Limiting)

The **Fixed Window Counter Algorithm** is one of the simplest and most common rate limiting techniques.

It divides time into fixed intervals and limits how many requests can happen inside each interval.

1. Core Idea

Split time into fixed windows such as:

- 1 second
- 1 minute
- 1 hour

Each window has a counter.

None

```
If requests in current window exceed limit,  
new requests are rejected until next window starts.
```

2. How It Works

Step 1: Divide Timeline into Fixed Windows

Examples:

None

`00:00:00 – 00:00:59`

`00:01:00 – 00:01:59`

`00:02:00 – 00:02:59`

Each window gets its own request counter.

Step 2: Increment Counter per Request

Every incoming request:

None

`counter = counter + 1`

Step 3: Enforce Threshold

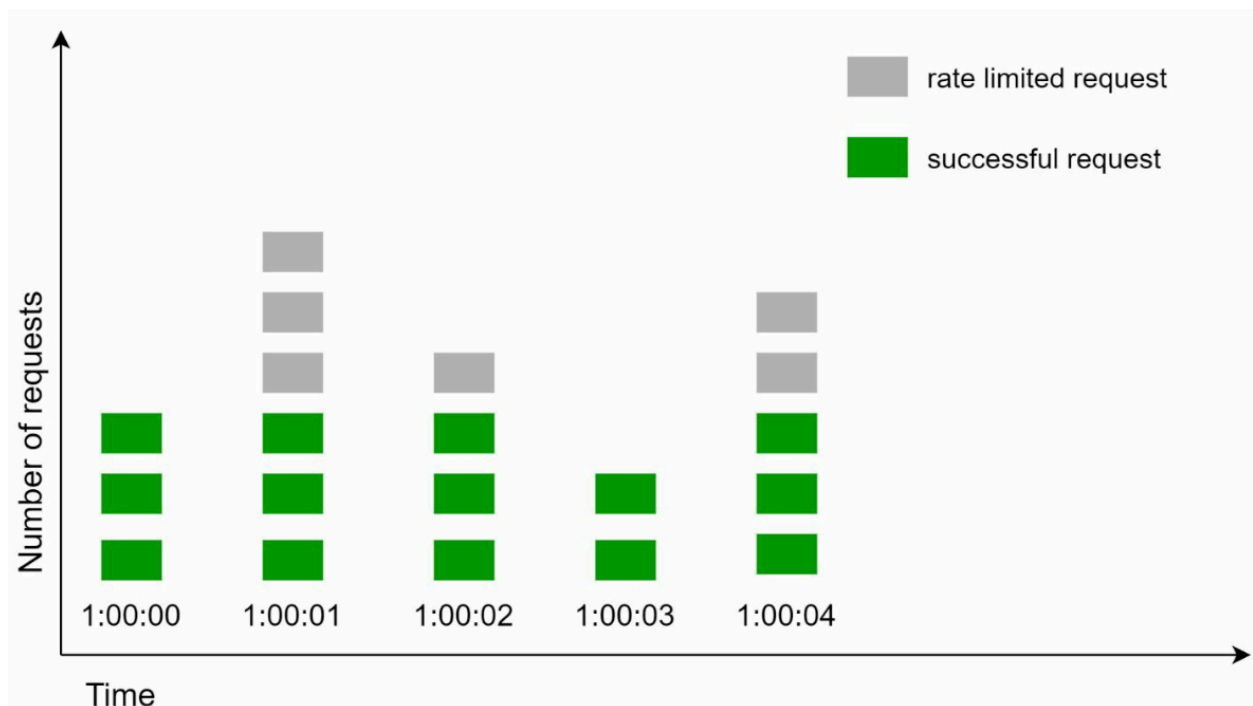
If counter reaches configured limit:

None

`Reject remaining requests in same window`

`Allow again when next window starts`

3. Example (Per Second Limit)



As shown in **Figure**:

System allows:

None

Max = 3 requests / second

Window:

None

10:00:01.000 → 10:00:01.999

Requests:

None

Req1 → allowed

Req2 → allowed

Req3 → allowed

Req4 → rejected

Req5 → rejected

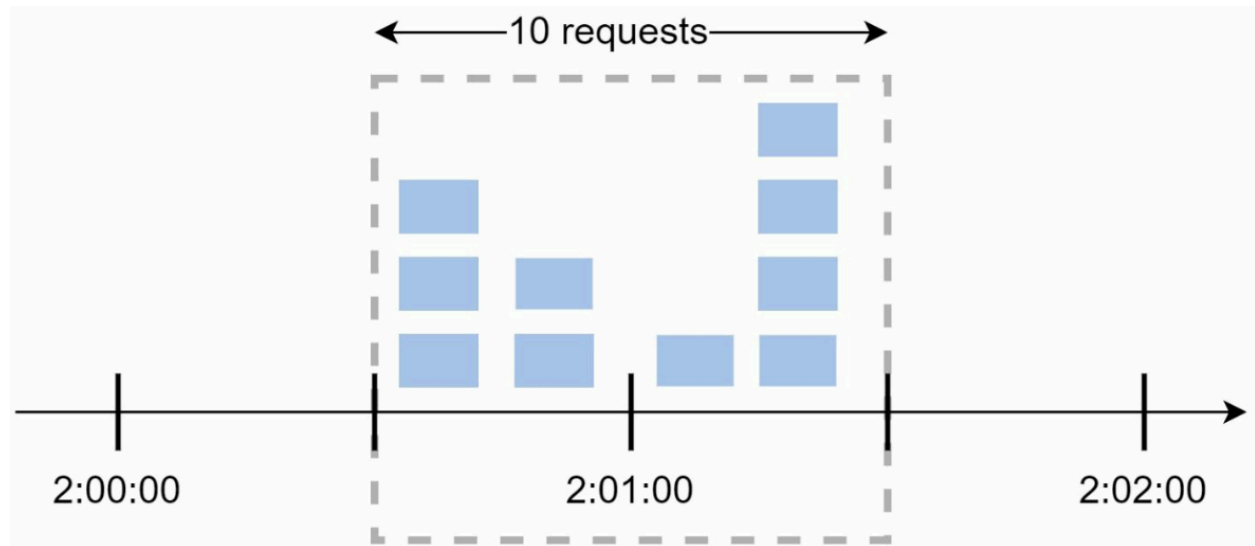
At next second:

None

Counter resets to 0

4. Major Problem: Boundary Burst Issue

This is the biggest weakness.



As shown in **Figure**:

System allows:

None

5 requests / minute

Traffic pattern:

None

5 requests at 2:00:59

5 requests at 2:01:00

Technically:

- First 5 belong to old window
- Next 5 belong to new window

So in only a few seconds:

None

10 requests passed

Even though effective intended rate was 5/minute.

5. Why It Is Popular

✓ Very Simple

Easy to understand and implement.

✓ Fast Performance

Only need one counter per key.

✓ Memory Efficient

Store:

- Current window timestamp
- Current count

6. Advantages

- Easy implementation
- Low memory usage
- Fast lookup/update
- Works well for coarse limits
- Useful where periodic reset is acceptable

7. Disadvantages

Window Edge Spikes

Burst traffic around reset boundary can bypass effective limit.

Less Accurate

Not smooth like sliding window.

Unfair Traffic Distribution

User can send many requests near boundary.

8. Example Use Cases

Good for:

- Simple public APIs
- Low-risk throttling
- Daily quota resets
- Internal tooling limits

Examples:

None

1000 requests/day

50 requests/minute

9. Implementation in Distributed Systems

Use shared counters in:

- Redis
- In-memory gateway counters
- API gateway plugins

Need atomic increment operations.

10. Comparison vs Better Alternatives

Algorithm	Accuracy	Burst Handling	Complexity
Fixed Window	Low	Poor	Very Low
Sliding Window	High	Better	Medium
Token Bucket	High	Excellent	Medium
Leaky Bucket	Medium	Smooth Output	Medium

Mental Model

None

Fixed Window = Count requests in time block, then reset

One-Line Summary

Fixed Window Counter Algorithm limits requests by counting traffic inside fixed time intervals, making it simple and efficient, but prone to burst spikes at window boundaries.